

BLOCKCHAIN & SMART CONTRACTS

Mini Uniswap

A Learning Guide

From zero to a working decentralized exchange — how it all works, and why it's built this way.

Prepared for Joshua Shabu · github.com/Joshua-Shabu/swapper
Covers: blockchain & Ethereum fundamentals, ERC-20, AMM math, Solidity, Hardhat 3, ethers.js

Contents

Contents

Mini Uniswap: A Blockchain & AMM DEX Learning Guide

How to use this document

Part 1 — Blockchain and Ethereum Fundamentals

1.1 What a blockchain actually is

1.2 Accounts, keys, and wallets

1.3 Transactions and gas

1.4 Smart contracts and the EVM

Part 2 — The ERC-20 Token Standard

2.1 Why a standard exists

2.2 Token.sol, in full

2.3 Walking through it

Part 3 — Automated Market Makers: The Core Idea

3.1 The problem: how do you set a price with no counterparty?

3.2 The constant-product formula

3.3 Where the fee fits in

3.4 Liquidity provider (LP) tokens

3.5 Impermanent loss (the risk side of the trade)

Part 4 — Pair.sol Deep Dive

4.1 State and setup

4.2 The reentrancy guard

4.3 `_update`, `_mintLP`/`_burnLP`, `_sqrt` — the private helpers

4.4 `mint` — depositing liquidity

4.5 `swap` — the trading function

4.6 `burn` — withdrawing liquidity

Part 5 — Factory.sol and CREATE2

Part 6 — Solidity Language Notes

Part 7 — Hardhat 3: Tooling and Workflow

7.1 What Hardhat actually does for you

7.2 Project structure

7.3 Compiling and testing

7.4 Deploying: scripts vs. Ignition

Part 8 — The Frontend: Connecting React to the Blockchain

8.1 The provider/signer split

8.2 Reading state: balances and reserves

8.3 Computing quotes locally

8.4 The two-transaction swap flow

8.5 Add liquidity, and the auto-suggest ratio

Part 9 — Project Architecture, End to End

Part 10 — What's Next

Glossary

Mini Uniswap: A Blockchain & AMM DEX Learning Guide

From zero to a working decentralized exchange — how it all works, and why it's built this way.

How to use this document

This is the reference for the project you built: **Mini Uniswap**, a simplified Uniswap V2-style automated market maker (AMM) with an ERC-20 token, a constant-product trading pair, a factory that deploys pairs, and a React frontend that talks to all of it through MetaMask. Every code sample in this document is the *actual* code in your project, not a simplified stand-in — so you can read this side by side with your editor.

The guide is organized to build understanding in layers: blockchain fundamentals first, then the token standard, then the AMM math, then the contracts themselves line by line, then the tooling, then the frontend. If a term feels unfamiliar, check the Glossary at the end.

Part 1 — Blockchain and Ethereum Fundamentals

1.1 What a blockchain actually is

Strip away the hype and a blockchain is a specific kind of database: an append-only log of transactions, replicated across thousands of independent computers ("nodes"), where every node agrees on the exact same history. Three properties fall out of that design:

- **Append-only.** You can add new transactions, but you can't rewrite old ones — each block cryptographically references the block before it (via a hash), so tampering with old history breaks every block after it.
- **No single owner.** Anyone can run a node. No company or person can unilaterally change the rules or reverse a transaction.
- **Publicly verifiable.** Anyone can independently check that the whole history is valid, without trusting whoever told them about it.

A **block** is just a batch of transactions plus a pointer (hash) to the previous block — that pointer chain is where the name "blockchain" comes from. Nodes need to agree on which transactions go in which block and in what order; the mechanism they use to reach that agreement without a central authority is called **consensus**. Ethereum (since "the Merge" in 2022) uses **proof-of-stake**: validators lock up ETH as collateral and take turns proposing blocks, and validators who propose invalid blocks lose (part of) their stake. Bitcoin, by contrast, uses **proof-of-work** (miners compete to solve a computational puzzle) — different consensus mechanism, same underlying goal.

1.2 Accounts, keys, and wallets

Ethereum has two kinds of accounts:

- **Externally Owned Accounts (EOAs)** — controlled by a private key. This is "your" account — MetaMask generates and holds a private key for you, and every transaction you send is cryptographically signed with it.
- **Contract accounts** — controlled by code, not a private key. Once deployed, a smart contract's logic decides what it will and won't do. Nobody "holds the keys" to a contract in the traditional sense (unless the contract's own code grants some address special privileges).

A private key is just a very large random number. From it, elliptic-curve cryptography derives a matching **public key**, and from that, an **address** (the `0x...` identifier you're used to seeing). The one-way nature of this derivation is the whole security model: anyone can see your address and public key, but only you can produce a valid signature for it, because only you have the private key. **MetaMask** is a wallet — software that stores your private key (encrypted, locally) and signs transactions on your behalf when you approve them. It never sends your private key anywhere; it sends a *signature*.

When you imported an account into MetaMask using one of the private keys Hardhat printed out (`npm hardhat node` lists 20 funded test accounts), you were doing exactly this: giving MetaMask a private key so it could sign transactions as that address.

1.3 Transactions and gas

Every write to the blockchain — sending ETH, deploying a contract, calling a contract function that changes state — is a **transaction**. Transactions aren't free: the network charges **gas**, a fee paid to whoever validates the block, priced in ETH. Gas exists for a very practical reason: without a cost, anyone could submit an infinite loop and grind the entire network to a halt. Gas makes computation on a shared, global computer economically bounded.

Two separate numbers make up a gas cost:

- **Gas units** — how much computational work the transaction does. A simple ETH transfer costs 21,000 gas; a contract deployment or a complex function call costs more, proportional to the work (storage writes are especially expensive — much more than reads).
- **Gas price** — how much you're willing to pay per unit, in a tiny denomination of ETH called **gwei** (1 ETH = 1,000,000,000 gwei). When the network is busy, gas prices rise because block space is a scarce, auctioned resource.

`Total fee = gas units used × gas price`. On your local Hardhat network, gas is still charged and tracked (you can watch account balances tick down slightly with each transaction), but since the ETH is fake and the "network" is just your own machine, it costs nothing real. On a public network like Sepolia or mainnet, real ETH is spent.

1.4 Smart contracts and the EVM

A **smart contract** is a program deployed to the blockchain at a fixed address, with logic that runs deterministically on every node in exact lockstep. Ethereum nodes execute contract code inside the **Ethereum Virtual Machine (EVM)** — a sandboxed, deterministic runtime, so that "run this contract" produces the exact same result on every node in the world, which is what lets them all agree on the outcome.

Contracts are usually written in a higher-level language — **Solidity** is by far the most common — and compiled down to EVM **bytecode**, the actual instructions stored on-chain. Alongside the bytecode, the compiler produces an **ABI (Application Binary Interface)** — a JSON description of the contract's functions and events, which is how external tools (like our frontend) know how to correctly encode a call to the contract and decode what comes back. You've been hand-writing trimmed ABIs yourself in `frontend/src/contracts.ts`.

The defining property of a deployed contract is **immutability**: once deployed, its code cannot be changed (barring specific upgrade patterns designed in ahead of time, which this project doesn't use). This is a feature and a risk at once — it's what makes a contract trustworthy (nobody, including its creator, can quietly change the

rules later), and it's exactly why smart contract bugs are so consequential: there's no patching a live, immutable contract that's already holding funds. It's also why testing (Part 7) matters so much more here than in typical web development.

Part 2 — The ERC-20 Token Standard

2.1 Why a standard exists

Before ERC-20 (proposed in 2015), every token contract on Ethereum could look completely different, so every wallet and exchange needed custom code for every token. ERC-20 fixed that by defining a minimal, common interface: implement these specific functions and events, and any wallet, exchange, or contract that speaks ERC-20 can work with your token automatically — including, eventually, our own AMM. It's the same idea as a USB port: a shared shape everyone can plug into.

2.2 Token.sol, in full

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Token {
    string public name;
    string public symbol;
    uint8 public decimals = 18;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(string memory _name, string memory _symbol, uint256 _initialSupply) {
        name = _name;
        symbol = _symbol;

        uint256 amount = _initialSupply * (10 ** uint256(decimals));
        totalSupply = amount;
        balanceOf[msg.sender] = amount;

        emit Transfer(address(0), msg.sender, amount);
    }

    function transfer(address to, uint256 value) external returns (bool) {
        _transfer(msg.sender, to, value);
        return true;
    }

    function approve(address spender, uint256 value) external returns (bool) {
        allowance[msg.sender][spender] = value;
        emit Approval(msg.sender, spender, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) external returns (bool) {
        uint256 allowed = allowance[from][msg.sender];
        require(allowed >= value, "ERC20: transfer exceeds allowance");
        if (allowed != type(uint256).max) {
            allowance[from][msg.sender] = allowed - value;
        }
        _transfer(from, to, value);
    }
}
```

```

        return true;
    }

    function _transfer(address from, address to, uint256 value) internal {
        require(to != address(0), "ERC20: transfer to zero address");
        uint256 fromBalance = balanceOf[from];
        require(fromBalance >= value, "ERC20: transfer exceeds balance");

        balanceOf[from] = fromBalance - value;
        balanceOf[to] += value;

        emit Transfer(from, to, value);
    }
}

```

Note this was deliberately hand-written from scratch, without OpenZeppelin's audited ERC-20 implementation, so you'd see exactly what the standard requires under the hood. (In anything beyond a learning project, you'd import OpenZeppelin's version instead of hand-rolling this — it's been audited by many independent security firms and used by thousands of production contracts; a hand-written token is a fine *pedagogical* exercise, and a real risk in production.)

2.3 Walking through it

State variables. `name`, `symbol`, and `decimals` are metadata — not strictly required by the ERC-20 spec, but every wallet and app expects them. `decimals = 18` is a convention (matching ETH itself): it means the contract's internal integer math is done in units of 10^{-18} tokens, so "1 token" is actually stored as `1_000_000_000_000_000_000`. This exists because the EVM has no native decimal/floating-point type — Solidity only has integers, so "decimal" tokens are really just integers with an agreed-upon scaling factor, and it's the *frontend's* job (via `ethers.formatUnits` / `parseUnits`) to convert between the human-readable "10.5 tokens" and the raw integer the contract actually stores.

`balanceOf` and `allowance` are mappings, Solidity's hash-table-like key → value storage type. `mapping(address => uint256) public balanceOf` is "give me an address, I'll give you its balance." `allowance` is a mapping of a mapping — `allowance[owner][spender]` — "how much of owner's tokens has owner said spender can move."

Events exist for anyone watching the chain, not for the contract itself. `emit Transfer(...)` doesn't do anything to the contract's own state — the actual balance changes already happened on the lines above it. Events write a permanent, cheap-to-store log entry that off-chain software (wallets, block explorers, our own frontend if we chose to listen for them) can subscribe to, so they can react to what happened without constantly polling contract storage.

transfer vs. approve / transferFrom — two different permission models. `transfer` moves *your own* tokens directly — simple. But our AMM contract needs to pull tokens *out of your wallet* when you swap or add liquidity, and it obviously can't do that with your private key. The `approve / transferFrom` pattern solves this: you (the owner) call `approve(spender, amount)` to pre-authorize another address to move up to `amount` of your tokens; that other contract then calls `transferFrom(you, someone, amount)` to actually move them, and the contract decrements your `allowance` as it spends it. This is the standard mechanism every DeFi protocol uses to interact with tokens it doesn't own. (Our own Pair contract's `mint / swap / burn` actually use the *simpler* transfer-then-call pattern instead of `transferFrom` — see Part 4.4 — but understanding `approve / allowance` is essential because it's how the vast majority of real DEXs, including full Uniswap, actually work.)

Checks before effects. `_transfer` checks `fromBalance >= value` *before* touching either balance. This ordering — validate first, then mutate state — is a basic but important defensive habit in Solidity, where a failed `require` cleanly reverts the *entire* transaction (all state changes in it are rolled back, as if it never happened), so a check partway through is still perfectly safe; the danger patterns are more subtle and are covered in Part 6.5 (reentrancy).

Solidity 0.8+ overflow/underflow checks are automatic. `balanceOf[from] = fromBalance - value` looks like it could underflow to a giant number if `value > fromBalance` — but it can't reach that line, because the `require` above already guarantees `fromBalance >= value`. And even if it *could* somehow be reached, Solidity $\geq 0.8.0$ (this project uses `^0.8.20`) reverts automatically on any arithmetic overflow or underflow, by default, with no extra code needed. This is a meaningful difference from Solidity 0.7 and earlier, where overflow silently wrapped around (`0 - 1` used to become the maximum `uint256` value!) unless you used a library like OpenZeppelin's `SafeMath`. Post-0.8, `SafeMath` is largely unnecessary for basic arithmetic.

Part 3 — Automated Market Makers: The Core Idea

3.1 The problem: how do you set a price with no counterparty?

A traditional exchange (Coinbase, NASDAQ, and yes, an early self-built version of Uniswap's *own* predecessor) uses an **order book**: buyers and sellers each post the price and quantity they want, and the exchange matches compatible orders. This requires *someone on the other side* willing to trade at your price, at that moment — which works fine for high-volume assets but breaks down badly for anything thin: you might want to sell a token nobody's currently trying to buy.

An **automated market maker** sidesteps this entirely. Instead of matching individual buyers with individual sellers, an AMM pools liquidity from many depositors into a shared reserve, and prices trades algorithmically based on the *ratio* of the two reserves — there's always something to trade against, as long as the pool has any liquidity at all, with no need for a matching counterparty in that instant.

3.2 The constant-product formula

Uniswap's innovation (and the mechanism our `Pair.sol` implements) is beautifully simple: for a pool holding reserve x of token A and reserve y of token B, the product $x \times y$ is kept constant ($= k$) across every trade, ignoring fees for a moment. If you want to buy some amount of token B, you have to deposit enough token A to keep that product the same:

$$x \cdot y = k \quad (\text{before trade: } x, y - \text{after trade: } x', y' - x' \cdot y' = k \text{ also})$$

Say a pool holds 10,000 A and 20,000 B, so $k = 200,000,000$. If you deposit 100 A (new $x' = 10,100$), the formula tells you exactly how much B you're owed to keep $x' \cdot y' = k$:

$$y' = k / x' = 200,000,000 / 10,100 \approx 19,801.98$$

You'd receive $20,000 - 19,801.98 \approx 198.02$ B for your 100 A. Notice you *didn't* get 200 B (the "even" ratio at 1 A = 2 B) — you got slightly less. That gap is **price impact** (also called **slippage**): the larger your trade is relative to the pool's size, the worse a rate you get, because you're moving the ratio (and therefore the implied price) as you trade. This is a *feature*, not a flaw — it's what makes the formula self-balancing: big trades naturally cost more per unit, which discourages draining a pool in one shot and gives arbitrageurs an incentive to correct any price that drifts away from the broader market.

3.3 Where the fee fits in

Real trading has to compensate liquidity providers for the risk they take on (see 3.5), so a small fee — 0.3%, the same rate Uniswap V2 uses — is taken out of every trade's input before the constant-product formula is applied. In code (from `getAmountOut` in the frontend, and mirrored inside `Pair.swap`):

```
function getAmountOut(amountIn: bigint, reserveIn: bigint, reserveOut: bigint): bigint {
  const amountInWithFee = amountIn * 997n;           // keep 99.7% of the input
  const numerator = amountInWithFee * reserveOut;
  const denominator = reserveIn * 1000n + amountInWithFee;
  return numerator / denominator;
}
```

Multiplying by 997 and dividing by 1000 elsewhere is just fixed-point math to express " $\times 0.997$ " without floating-point numbers, which the EVM (and, by extension, Solidity) doesn't have at all — *every* number in Solidity is an integer, so any "decimal" behavior anywhere in these contracts is really careful integer arithmetic wearing a decimal costume. The 0.3% that's held back doesn't go anywhere special — it simply stays in the pool, permanently growing the reserves (and therefore `k`) a little with every trade. That's *how* liquidity providers earn a return: their share of an ever-so-slightly-growing pool.

3.4 Liquidity provider (LP) tokens

When you deposit into a pool, you receive **LP tokens** representing your proportional share of it — in this project, the `Pair` contract is *itself* an ERC-20-like token (see `name = "Mini Uniswap LP"`, `symbol = "MINI-LP"` in `Pair.sol`), exactly matching how real Uniswap V2 works. Holding more LP tokens means you own a bigger slice of the pool's *current* reserves (which include all the fees collected since you deposited); burning your LP tokens back gives you your proportional share of whatever's in the pool at that moment — not necessarily the same ratio of A:B you originally put in, because the ratio moves with every trade.

3.5 Impermanent loss (the risk side of the trade)

Because arbitrageurs keep the pool's ratio in line with the wider market price by trading against it, a liquidity provider's holdings drift as the price of the two tokens moves relative to each other — if token A rises sharply against token B, arbitrage trades will have pulled A *out* of the pool and pushed B *in*, so by the time you withdraw, you'll have less A and more B than a simple "just held both tokens" strategy would have left you with. This gap between "value if you'd just held" and "value if you provided liquidity" is called **impermanent loss** — "impermanent" because it only becomes a *realized* loss if you withdraw while the price ratio is skewed; if the ratio returns to where it started, so does your position. It's the central risk LPs take on in exchange for earning trading fees, and it's larger the more the two tokens' prices move relative to each other (pairing two assets that tend to move together, like two different USD stablecoins, minimizes it; pairing volatile, uncorrelated assets maximizes it).

Part 4 — `Pair.sol` Deep Dive

This is the entire trading engine of the DEX — everything else (the token, the factory, the frontend) exists to support what happens in this one contract.

4.1 State and setup

```
uint256 public constant MINIMUM_LIQUIDITY = 1000;

address public token0;
address public token1;

uint112 private reserve0;
uint112 private reserve1;

string public constant name = "Mini Uniswap LP";
string public constant symbol = "MINI-LP";
uint256 public totalSupply;
mapping(address => uint256) public balanceOf;
```

`reserve0` / `reserve1` are `uint112` rather than the more common `uint256` — a deliberate real-Uniswap-V2 optimization. The EVM's storage is organized into 32-byte ("word") slots, and reading/writing storage is by far the most expensive operation a contract does. Packing `reserve0`, `reserve1`, and a timestamp (which this simplified version omits, but real Uniswap V2 includes for its price-oracle feature) into a *single* 32-byte storage slot instead of three separate ones cuts gas costs meaningfully on every trade, since it's one storage write instead of two or three. `uint112` tops out at about 5.2×10^{33} — vastly more than any real token supply — so nothing is lost by using a smaller type here.

`MINIMUM_LIQUIDITY = 1000` defends against a specific first-depositor attack: without it, the very first liquidity provider could deposit a tiny amount (getting a correspondingly tiny number of LP tokens, even down to 1), then directly transfer a huge amount of tokens into the pool without minting more LP tokens for it, wildly inflating the value of each existing LP token — before a second, unsuspecting depositor comes in and gets rounded down to zero new LP tokens for a real deposit, effectively donating their funds to the first depositor. Permanently burning a fixed 1000 units of the very first mint (sent to `address(0xdead)`, an address nobody controls) makes this attack economically pointless for any realistic pool size.

4.2 The reentrancy guard

```
bool private locked;
modifier lock() {
    require(!locked, "Pair: LOCKED");
    locked = true;
    _;
    locked = false;
}
```

A **modifier** in Solidity wraps a function with reusable pre/post logic — `_;` marks where the wrapped function's own body runs. `lock` implements the classic **reentrancy guard**: a boolean flag that's flipped on at the start of a protected function and off at the end, with a `require` at the top refusing to let the function run again while it's already "inside" a call.

Why this matters: `mint`, `burn`, and `swap` all call out to *external* contracts (the two ERC-20 tokens) mid-function — for example, `swap` calls `token0.transfer(...)` before finishing its own accounting. If that external call could somehow call *back* into the `Pair` contract before the outer call finishes (this is easy if the "token" is actually malicious code, or even just a token with unusual callback hooks) it could see the contract in an inconsistent, half-updated state and exploit it — this exact bug pattern was responsible for The DAO hack in 2016, one of the most consequential incidents in Ethereum's history, and reentrancy guards have been standard defensive practice ever since. Every state-changing function in `Pair.sol` (`mint`, `burn`, `swap`) is marked `lock`.

4.3 `_update`, `_mintLP` / `_burnLP`, `_sqrt` — the private helpers

```
function _update(uint256 balance0, uint256 balance1) private {
    require(balance0 <= type(uint112).max && balance1 <= type(uint112).max, "Pair: OVERFLOW");
    reserve0 = uint112(balance0);
    reserve1 = uint112(balance1);
    emit Sync(reserve0, reserve1);
}
```

`_update` is the single place reserves actually get written — every operation ends by calling it with the pool's *actual current* token balances (read fresh from the token contracts), rather than trying to track running totals through arithmetic. This "trust the actual balance, not a running counter" approach is deliberate: it makes the reserves self-correcting against any accounting drift, and it's *why* the transfer-then-call pattern below works at all — the contract literally checks its own balance to see what arrived.

`_sqrt` implements the **Babylonian method** (a form of Newton's method) for integer square roots — used exactly once, to size the very first LP mint. The reason a square root shows up at all: minting $\sqrt{\text{amount0} \times \text{amount1}}$ LP tokens on the first deposit is what makes the *initial* price you happen to set (the ratio of `amount0`

to amount1) not bias how many LP tokens you receive — only the geometric-mean "size" of the deposit does. The EVM has no built-in square root operation (or floating point at all), so this loop-based integer approximation is the standard way to compute one on-chain.

4.4 `mint` — depositing liquidity

```
function mint(address to) external lock returns (uint256 liquidity) {
    (uint112 _reserve0, uint112 _reserve1) = getReserves();
    uint256 balance0 = IERC20Minimal(token0).balanceOf(address(this));
    uint256 balance1 = IERC20Minimal(token1).balanceOf(address(this));
    uint256 amount0 = balance0 - _reserve0;
    uint256 amount1 = balance1 - _reserve1;

    if (totalSupply == 0) {
        liquidity = _sqrt(amount0 * amount1) - MINIMUM_LIQUIDITY;
        _mintLP(address(0xdead), MINIMUM_LIQUIDITY);
    } else {
        liquidity = _min((amount0 * totalSupply) / _reserve0, (amount1 * totalSupply) / _reserve1);
    }
    require(liquidity > 0, "Pair: INSUFFICIENT_LIQUIDITY_MINTED");
    _mintLP(to, liquidity);

    _update(balance0, balance1);
    emit Mint(msg.sender, amount0, amount1);
}
```

The single most important thing to notice here: `mint` never calls `transferFrom`. It assumes the caller *already* sent token0 and token1 directly to the Pair contract's own address, and figures out how much arrived by comparing the current balance against the last-recorded reserve (`amount0 = balance0 - _reserve0`). This is the **transfer-then-call pattern**, and it's exactly what real Uniswap V2's core does too (a separate "Router" contract normally bundles the transfer and the mint call into one atomic user-facing transaction — this project's frontend does that bundling itself in JavaScript instead, as two sequential transactions; see Part 8.4).

For subsequent deposits (`totalSupply != 0`), notice the `_min(...)` — you're credited LP tokens based on whichever of the two tokens you contributed *proportionally less* of, protecting existing LPs from a depositor trying to sneak in an imbalanced, undervalued deposit and walk away with an outsized share.

4.5 swap — the trading function

```
function swap(uint256 amount0Out, uint256 amount1Out, address to) external lock {
    require(amount0Out > 0 || amount1Out > 0, "Pair: INSUFFICIENT_OUTPUT_AMOUNT");
    (uint112 _reserve0, uint112 _reserve1) = getReserves();
    require(amount0Out < _reserve0 && amount1Out < _reserve1, "Pair: INSUFFICIENT_LIQUIDITY");

    if (amount0Out > 0) IERC20Minimal(token0).transfer(to, amount0Out);
    if (amount1Out > 0) IERC20Minimal(token1).transfer(to, amount1Out);

    uint256 balance0 = IERC20Minimal(token0).balanceOf(address(this));
    uint256 balance1 = IERC20Minimal(token1).balanceOf(address(this));

    uint256 amount0In = balance0 > _reserve0 - amount0Out ? balance0 - (_reserve0 - amount0Out) : 0;
    uint256 amount1In = balance1 > _reserve1 - amount1Out ? balance1 - (_reserve1 - amount1Out) : 0;
    require(amount0In > 0 || amount1In > 0, "Pair: INSUFFICIENT_INPUT_AMOUNT");

    uint256 balance0Adjusted = balance0 * 1000 - amount0In * 3;
    uint256 balance1Adjusted = balance1 * 1000 - amount1In * 3;
    require(
        balance0Adjusted * balance1Adjusted >= uint256(_reserve0) * uint256(_reserve1) * 1000
        "Pair: K"
    );

    _update(balance0, balance1);
    emit Swap(msg.sender, amount0In, amount1In, amount0Out, amount1Out, to);
}
```

Notice the *order* of operations: `swap` sends the output tokens out **first**, then figures out what came in by re-checking balances, then verifies the invariant held. This looks backwards until you remember it's built for the same transfer-then-call world as `mint`: it doesn't ask "did you pay me first?" — it asks "after everything moved, does the math still check out?" — and reverts the *entire* transaction if not (recall: a failed `require` unwinds everything, including the token transfers that already happened inside this same call). The frontend, correspondingly, always sends the input tokens to the pool *before* calling `swap` (see Part 8.4) — but the contract itself doesn't strictly require that ordering; it only requires that by the time `swap` finishes executing, enough of *something* came in to satisfy the invariant check below.

The invariant check is the fee-adjusted constant-product formula from Part 3.3, verified directly in integer math (no separate `getAmountOut` call inside the contract — the caller is trusted to have already computed a valid `amount0Out / amount1Out` off-chain, and this line is what actually *enforces* that they got the math right, on-chain, regardless of what the frontend calculated). If `balance0Adjusted * balance1Adjusted` comes up short of `reserve0 * reserve1 * 1000 * 1000`, the whole trade reverts — there's no way to under-pay the pool.

4.6 burn — withdrawing liquidity

```
function burn(address to) external lock returns (uint256 amount0, uint256 amount1) {
    uint256 balance0 = IERC20Minimal(token0).balanceOf(address(this));
    uint256 balance1 = IERC20Minimal(token1).balanceOf(address(this));
    uint256 liquidity = balanceOf[address(this)];

    amount0 = (liquidity * balance0) / totalSupply;
    amount1 = (liquidity * balance1) / totalSupply;
    require(amount0 > 0 && amount1 > 0, "Pair: INSUFFICIENT_LIQUIDITY_BURNED");

    _burnLP(address(this), liquidity);
    IERC20Minimal(token0).transfer(to, amount0);
    IERC20Minimal(token1).transfer(to, amount1);
    ...
}
```

Same transfer-then-call shape as `mint`, in reverse: the caller sends their *LP tokens* to the Pair contract's own address first (`balanceOf[address(this)]` — the Pair checking its own LP token balance of itself), then calls `burn`, which reads how many LP tokens arrived, calculates the proportional share of both reserves, and sends both tokens out. This function exists in the deployed contract and is fully functional, but the frontend doesn't have a "Remove Liquidity" button wired up yet — a natural next feature to add (see Part 10).

Part 5 — `Factory.sol` and `CREATE2`

```
contract Factory {
    mapping(address => mapping(address => address)) public getPair;
    address[] public allPairs;

    event PairCreated(address indexed token0, address indexed token1, address pair, uint256 pa

    function createPair(address tokenA, address tokenB) external returns (address pair) {
        require(tokenA != tokenB, "Factory: IDENTICAL_ADDRESSES");

        (address token0, address token1) = tokenA < tokenB ? (tokenA, tokenB) : (tokenB, tokenA);
        require(token0 != address(0), "Factory: ZERO_ADDRESS");
        require(getPair[token0][token1] == address(0), "Factory: PAIR_EXISTS");

        bytes32 salt = keccak256(abi.encodePacked(token0, token1));
        pair = address(new Pair{salt: salt}(token0, token1));

        getPair[token0][token1] = pair;
        getPair[token1][token0] = pair;
        allPairs.push(pair);

        emit PairCreated(token0, token1, pair, allPairs.length - 1);
    }
}
```

Sorting the addresses (`tokenA < tokenB`, comparing them as numbers — Solidity `address` values support ordering comparisons for exactly this purpose) guarantees that asking for a "TKA/TKB" pair and a "TKB/TKA" pair always resolves to the same underlying `Pair` contract, regardless of which order a caller happens to pass them in — without this, it would be possible to accidentally fragment liquidity for the same pair of tokens across two separate, uncoordinated pools.

CREATE2 is the more interesting piece. Normally, when a contract deploys another contract (`new Pair(...)`), the new contract's address is derived from the *deployer's address and its transaction nonce* — meaning you can't know the address in advance without knowing exactly how many transactions the deployer will have sent by the time it happens. `CREATE2` (invoked here via the `{salt: salt}` syntax) instead derives the address deterministically from the deployer's address, a `salt` value the deployer chooses, and the *bytecode* of the contract being deployed — which means the resulting address is fully computable **off-chain, before the pair is ever created**, given nothing but the `Factory`'s address, the two token addresses, and the `Pair` contract's bytecode. Real Uniswap relies on this so that other contracts (like a router that needs to interact with many pairs) can compute a pair's address directly with a formula, rather than needing to call back to the `Factory` and ask "what's

the address for this pair?" every single time — a meaningful gas and complexity savings at scale. Here, the `salt` is simply `keccak256(token0, token1)` — deterministic and unique per token pair, which is exactly what's needed.

`getPair` is a nested mapping written in both directions (`getPair[token0][token1]` and `getPair[token1][token0]`) purely for lookup convenience — so a caller can query with tokens in *either* order and get the right answer, without needing to replicate the Factory's own sorting logic just to ask a question.

Part 6 — Solidity Language Notes

A concentrated reference for the language-level concepts that show up throughout `Token.sol`, `Pair.sol`, and `Factory.sol` — aimed at someone comfortable in TypeScript/JavaScript encountering Solidity's differences for the first time.

6.1 Everything is an integer. There is no native floating-point or decimal type. `uint256` (0 to $\sim 1.15 \times 10^{77}$), `uint112`, `uint8`, and their signed counterparts (`int256`, etc.) are the numeric types you'll see. "Decimals" are a *convention*, not a language feature — see Part 2.3.

6.2 storage vs. memory vs. calldata. Solidity is explicit about *where* data lives, because storage (the blockchain's permanent state) is drastically more expensive to read and write than transient memory. State variables (declared at contract level, like `balanceOf`) always live in `storage`. Local variables inside a function default differently depending on type — value types (`uint256`, `bool`, `address`) are always just copied around cheaply; but for reference types (`string`, arrays, structs), you'll see explicit annotations like `string memory _name` in `Token.sol`'s constructor — meaning "keep a temporary, function-scoped copy," as opposed to `storage` (a permanent, persistent reference) or `calldata` (a read-only, even-cheaper location used for external function *parameters* specifically, since they never need to be modified).

6.3 Function visibility: external, public, internal, private. These aren't just documentation — they're enforced by the compiler. `external` functions can only be called from outside the contract (this is slightly cheaper in gas than `public` for functions never called internally, which is why `transfer`, `approve`, etc. are marked `external` rather than `public` throughout this project). `public` functions can be called from anywhere, inside or outside. `internal` functions (like `Token._transfer`) can only be called from within the contract itself or contracts that inherit from it. `private` (like all of `Pair`'s helper functions) is the strictest — callable only from within that exact contract, not even by an inheriting one. Note that "private" here means "not directly *callable* by outside code" — it does **not** mean the underlying data is secret. All contract storage is publicly readable on-chain by anyone willing to inspect it directly (block explorers, or raw RPC calls); `private` / `internal` are about controlling *access paths* in the code, never about confidentiality.

6.4 view and pure. `view` functions (like `getReserves`, `balanceOf`) promise not to modify any state — they can be called for free, without sending a transaction or paying gas, when queried directly (this is exactly how the frontend reads balances and reserves — no MetaMask popup, no gas, no waiting for a block). `pure` functions (like `_sqrt`, `_min`) go further: they promise not to even *read* contract state, only operate on their inputs.

6.5 Reentrancy, in one sentence. Any function that calls out to another contract mid-execution creates a window where that other contract's code runs *before* your function has finished — if it can call back into your

still-mid-update contract, it can see and exploit an inconsistent state; the `lock` modifier in Part 4.2 is the standard defense.

6.6 `require`, `revert`, and gas refunds. `require(condition, "message")` checks a condition and, if false, immediately reverts the entire transaction — every state change made so far in that transaction is undone, as though it never happened, and the revert message is returned to whoever called it (this is exactly what shows up as an error message in MetaMask or in your frontend's `catch` blocks). Gas already *spent* up to the point of the revert is not refunded — this is deliberate, so that a `require` failing late in a complex transaction still costs the sender something, which prevents cheap denial-of-service spam against expensive-to-simulate functions.

6.7 Events are for off-chain consumers, not on-chain logic. Covered in Part 2.3 — worth repeating here because it's a common point of confusion: no contract can ever read another contract's *past* emitted events. They exist purely in the transaction log for external software to watch.

Part 7 — Hardhat 3: Tooling and Workflow

7.1 What Hardhat actually does for you

Hardhat is the development environment tying everything together: it compiles Solidity down to bytecode + ABI, runs a local Ethereum node for fast iteration (`npx hardhat node`), runs your test suite against that (or an in-process) node, and deploys contracts to any network you configure — local, a public testnet, or mainnet — all through one consistent toolchain and configuration file (`hardhat.config.ts`).

This project uses **Hardhat 3**, a meaningfully different major version from the Hardhat 2 tutorials still most common online as of this writing — several CLI commands and conventions changed. The differences you hit firsthand, and their fixes:

What you tried	What Hardhat 3 actually wants
<code>npx hardhat init</code>	<code>npx hardhat --init</code>
<code>--network hardhat</code>	<code>--network hardhatMainnet</code> (the default in-memory network is named <code>hardhatMainnet</code> , not <code>hardhat</code>)
<code>npx hardhat test</code> (expecting the TS suite)	<code>npx hardhat test mocha</code> — Hardhat 3 splits Solidity tests and Mocha/TS tests into separate subcommands
<code>ethers</code> imported globally	<code>const { ethers } = await hre.network.create();</code> — connected explicitly per-network, inside an async context

7.2 Project structure

```
contracts/
├─ contracts/      # Solidity source: Token.sol, Pair.sol, Factory.sol
├─ test/           # Mocha + Chai + ethers.js tests (TypeScript)
├─ scripts/        # Imperative deploy scripts (deploy-local.ts)
├─ ignition/modules/ # Declarative Hardhat Ignition deployment modules
├─ hardhat.config.ts # Networks, Solidity compiler version, plugins
└─ artifacts/, cache/ # Build output (gitignored – regenerated by `npx hardhat compile`)
```

7.3 Compiling and testing

`npx hardhat compile` invokes the Solidity compiler (`solc`, version pinned in `hardhat.config.ts` — `0.8.34` for this project) and produces the ABI + bytecode for every contract, cached in `artifacts/`. `npx hardhat test mocha` runs the TypeScript test suite — here's the actual `Pair` test in full:

```

import { expect } from "chai";
import hre from "hardhat";

const { ethers } = await hre.network.create();

describe("Pair", function () {
  it("lets you add liquidity and then swap", async function () {
    const [deployer] = await ethers.getSigners();

    const tokenA = await ethers.deployContract("Token", ["Token A", "TKA", 1_000_000]);
    const tokenB = await ethers.deployContract("Token", ["Token B", "TKB", 1_000_000]);
    const pair = await ethers.deployContract("Pair", [
      await tokenA.getAddress(),
      await tokenB.getAddress(),
    ]);
    const pairAddress = await pair.getAddress();

    const amountA = ethers.parseUnits("10000", 18);
    const amountB = ethers.parseUnits("20000", 18);
    await tokenA.transfer(pairAddress, amountA);
    await tokenB.transfer(pairAddress, amountB);
    await pair.mint(deployer.address);

    const lpBalance = await pair.balanceOf(deployer.address);
    expect(lpBalance).to.be.gt(0n);

    let [reserveA, reserveB] = await pair.getReserves();
    expect(reserveA).to.equal(amountA);
    expect(reserveB).to.equal(amountB);

    const amountIn = ethers.parseUnits("100", 18);
    const amountInWithFee = (amountIn * 997n) / 1000n;
    const expectedOut = (amountInWithFee * reserveB) / (reserveA + amountInWithFee);

    await tokenA.transfer(pairAddress, amountIn);
    await pair.swap(0, expectedOut, deployer.address);

    [reserveA, reserveB] = await pair.getReserves();
    expect(reserveA).to.equal(amountA + amountIn);
    expect(reserveB).to.equal(amountB - expectedOut);
  });
});

```

Notice this test **independently recomputes** the expected swap output using the same 997/1000 fee formula, rather than just asserting "the contract didn't throw" — that's what actually proves the contract's math matches

the intended formula exactly, not merely that it runs without error. This "compute the expected result independently, then compare" pattern is worth carrying into every test you write from here on — a test that only checks "it didn't revert" catches far fewer bugs than one that checks the *specific number* it should have produced.

7.4 Deploying: scripts vs. Ignition

Hardhat offers two deployment approaches. **Ignition modules** (`ignition/modules/`) are declarative — you describe *what* should exist and Ignition figures out the ordering, and it tracks deployment state so a partially-failed deploy can resume rather than restart. They're the more "batteries included" option for straightforward deployments. **Imperative scripts** (`scripts/`), like this project's `deploy-local.ts` , are plain async functions with full manual control — useful once a deployment needs custom logic beyond "deploy these contracts," such as this project's need to seed the pool with initial liquidity in the same script:

```

import hre from "hardhat";

async function main() {
  const { ethers } = await hre.network.create();
  const [deployer] = await ethers.getSigners();

  const tokenA = await ethers.deployContract("Token", ["Mini Token A", "MTA", 1_000_000]);
  await tokenA.waitForDeployment();
  const tokenB = await ethers.deployContract("Token", ["Mini Token B", "MTB", 1_000_000]);
  await tokenB.waitForDeployment();

  const factory = await ethers.deployContract("Factory");
  await factory.waitForDeployment();

  const addrA = await tokenA.getAddress();
  const addrB = await tokenB.getAddress();

  const createTx = await factory.createPair(addrA, addrB);
  await createTx.wait();
  const pairAddress = await factory.getPair(addrA, addrB);
  const pair = await ethers.getContractAt("Pair", pairAddress);

  const amountA = ethers.parseUnits("10000", 18);
  const amountB = ethers.parseUnits("20000", 18);
  await (await tokenA.transfer(pairAddress, amountA)).wait();
  await (await tokenB.transfer(pairAddress, amountB)).wait();
  await (await pair.mint(deployer.address)).wait();

  console.log(`Token A: ${addrA}`);
  console.log(`Token B: ${addrB}`);
  console.log(`Factory: ${await factory.getAddress()}`);
  console.log(`Pair:    ${pairAddress}`);
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});

```

Every `await ...waitForDeployment()` / `await tx.wait()` matters: sending a transaction only *submits* it to the network; it isn't confirmed (mined into a block) yet when the `send` call returns. `wait()` pauses execution until the transaction is actually confirmed — skip it, and the very next line might try to interact with a contract that, from the network's point of view, doesn't exist yet.

This script produced the addresses currently deployed on your local network:

- **Token A (MTA):** `0x5FbDB2315678afecb367f032d93F642f64180aa3`
- **Token B (MTB):** `0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512`
- **Factory:** `0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0`
- **Pair:** `0xcF70CcAd086e02445Dc6eEf13E8b030e8775bC54`

(These addresses are only stable on a *fresh* local node deploying in this exact order — Hardhat's local network assigns addresses deterministically from the deployer's transaction nonce, so redeploying from a freshly restarted node in the same sequence reproduces the same addresses; deploying to a public network like Sepolia will produce entirely different ones.)

Part 8 — The Frontend: Connecting React to the Blockchain

8.1 The provider/signer split

`ethers.js` (v6, used throughout) draws a sharp line between two roles: a **provider** is a *read-only* connection to the blockchain — for querying balances, calling `view` functions, reading past events — and a **signer** is a provider *plus* the ability to sign and send transactions on behalf of one specific account. In the browser, `BrowserProvider(window.ethereum)` wraps whatever wallet extension is installed (MetaMask injects `window.ethereum`), and `provider.getSigner()` asks that wallet for permission to sign as the currently-selected account — which is exactly the moment MetaMask's popup appears asking you to connect or confirm.

8.2 Reading state: balances and reserves

```
async function refresh(address: string) {
  const provider = new BrowserProvider(window.ethereum);

  const balanceWei = await provider.getBalance(address);
  setEthBalance(formatUnits(balanceWei, 18));

  const tokenA = new Contract(TOKEN_A_ADDRESS, ERC20_ABI, provider);
  const tokenB = new Contract(TOKEN_B_ADDRESS, ERC20_ABI, provider);
  const [balA, balB] = await Promise.all([tokenA.balanceOf(address), tokenB.balanceOf(address)]);
  setBalances({ a: formatUnits(balA, 18), b: formatUnits(balB, 18) });

  const pair = new Contract(PAIR_ADDRESS, PAIR_ABI, provider);
  const [token0, reserveData, lpBal] = await Promise.all([
    pair.token0(),
    pair.getReserves(),
    pair.balanceOf(address),
  ]);
  const isAToken0 = token0.toLowerCase() === TOKEN_A_ADDRESS.toLowerCase();
  setToken0IsA(isAToken0);
  ...
}
```

Every one of these calls (`balanceOf`, `getReserves`, `token0`) is a `view` function (Part 6.4) — no transaction, no gas, no MetaMask popup, just an instant read. `new Contract(address, abi, provider)` constructs a JavaScript object whose methods are generated *from the ABI array* — this is why `frontend/src/contracts.ts` needs an accurate ABI: ethers uses it to know a function like `balanceOf` exists and how to encode/decode calling it, and a missing entry (as you discovered firsthand when

`tokenIn.transfer` turned out to be missing from `ERC20_ABI`) produces a hard runtime error the moment that method is called.

`token0IsA` deserves a specific callout, because it's a subtle but important piece of correctness: recall from Part 5 that `Factory.createPair` sorts the two token addresses before deploying — so which token ends up as the Pair's `token0` versus `token1` is **not** guaranteed to match the order you happen to call them "Token A" and "Token B" in your own frontend code. The app queries `pair.token0()` and compares it against the known `TOKEN_A_ADDRESS` at runtime, rather than assuming an order — every place that later needs to map "the token I'm calling A" onto "amount0 vs. amount1" (in both the swap and add-liquidity flows) reads from this flag instead of hardcoding an assumption. Skipping this check is a very easy, very silent way to swap the two tokens' amounts and send a transaction that behaves nothing like what the UI displayed.

8.3 Computing quotes locally

```
function getAmountOut(amountIn: bigint, reserveIn: bigint, reserveOut: bigint): bigint {
  const amountInWithFee = amountIn * 997n;
  const numerator = amountInWithFee * reserveOut;
  const denominator = reserveIn * 1000n + amountInWithFee;
  return numerator / denominator;
}
```

The frontend recomputes the exact same constant-product-with-fee formula the contract itself enforces (Part 4.5), so it can show you a live "you receive (estimated)" quote as you type, without needing a round trip to the blockchain. Note the `n` suffix on every literal (`997n` , `1000n`) — this is a JavaScript **BigInt** literal, required because token amounts routinely exceed `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) once scaled up by 10^{18} for 18 decimals; ordinary JavaScript numbers would silently lose precision on values this large, so `ethers` represents all on-chain integers as `bigint` , and mixing a `bigint` with a regular `number` in an arithmetic expression is a `TypeError` in JavaScript — deliberately, to stop exactly this kind of silent precision loss.

8.4 The two-transaction swap flow

```

async function handleSwap() {
  ...
  const tokenIn = new Contract(tokenInAddress, ERC20_ABI, signer);
  const pair = new Contract(PAIR_ADDRESS, PAIR_ABI, signer);

  setStatus("Sending tokens to the pool... (check MetaMask)");
  const tx1 = await tokenIn.transfer(PAIR_ADDRESS, amountInWei);
  await tx1.wait();

  const outIsToken0 = direction === "AtoB" ? !token0IsA : token0IsA;
  const amount0Out = outIsToken0 ? amountOutWei : 0n;
  const amount1Out = outIsToken0 ? 0n : amountOutWei;

  setStatus("Swapping... (check MetaMask)");
  const tx2 = await pair.swap(amount0Out, amount1Out, account);
  await tx2.wait();

  setStatus("Swap complete!");
  ...
}

```

This is the frontend's half of the transfer-then-call pattern from Part 4.4/4.5: two separate, sequential MetaMask-confirmed transactions — first a plain `transfer` sending your input tokens directly to the Pair contract's address, then a `swap` call telling the Pair how much of the *other* token to send back. Real Uniswap avoids exposing this two-step dance to end users by routing everything through a "Router" contract that can call `transfer` and `swap` together atomically in one on-chain transaction (still two calls under the hood, but bundled by a smart contract rather than by the browser). Doing it as two separate wallet-confirmed transactions here is simpler to build and — critically — **fully visible**: you can watch each step happen individually, which is exactly what makes it a good teaching version of the flow, at the cost of needing two MetaMask confirmations and, in a real setting, two separate gas payments instead of one.

Notice `outIsToken0` reuses the `token0IsA` flag from 8.2 to correctly map "the token I'm receiving" onto `amount0Out` vs. `amount1Out` — this is the exact place a token-order mixup would surface as a bug if that flag weren't tracked correctly.

8.5 Add liquidity, and the auto-suggest ratio

```
function handleAmountAChange(value: string) {
  setLiqAmountA(value);
  if (reserves.a > 0n && value) {
    const amountAWei = parseUnits(value, 18);
    const suggestedB = (amountAWei * reserves.b) / reserves.a;
    setLiqAmountB(formatUnits(suggestedB, 18));
  }
}
```

As you type an amount of Token A, the UI auto-fills a matching amount of Token B at the *pool's current ratio* — this is a UX nicety, not something the contract requires (recall `Pair.mint`, Part 4.4, will accept any two amounts you actually transfer in). It matters because depositing at an *imbalanced* ratio doesn't just look wrong — the contract's `_min(...)` logic means you'd only be credited LP tokens for the smaller, proportionally-matching side, effectively donating the excess. The add-liquidity flow itself is a three-transaction version of the same transfer-then-call pattern: transfer Token A in, transfer Token B in, then call `pair.mint(account)`.

Part 9 — Project Architecture, End to End

```

cryptoPro/
├── contracts/                # Hardhat 3 project
│   ├── contracts/
│   │   ├── Token.sol        # ERC-20 token
│   │   ├── Pair.sol         # Constant-product AMM pair (the trading engine)
│   │   └── Factory.sol      # Deploys + registers Pair contracts via CREATE2
│   └── test/                # Pair.ts, Factory.ts – Mocha/Chai/ethers tests
├── scripts/deploy-local.ts  # Deploys Token A/B + Factory, creates a pair, seeds liquidity
├── hardhat.config.ts        # Solidity 0.8.34, localhost + sepolia network configs
└── frontend/               # Vite + React + TypeScript
    └── src/
        ├── contracts.ts      # Deployed addresses + hand-written ABIs
        ├── App.tsx           # Wallet connect, swap UI, add-liquidity UI
        └── App.css           # Custom dark-theme design

```

The full round trip, for a swap: you type an amount → the frontend calls `getAmountOut` locally to show you a live quote (8.3) → you click Swap → MetaMask asks you to sign a `transfer` sending your input token to the Pair's address (8.4) → once confirmed, MetaMask asks you to sign a `swap` call → the Pair contract (4.5) checks its own balances, computes what actually arrived, verifies the constant-product invariant still holds after the fee, sends your output token, and updates its stored reserves → the frontend calls `refresh()` (8.2) to pull the new balances and reserves straight from the chain and re-render.

Every deployed address currently in `frontend/src/contracts.ts` points at your **local Hardhat network only** — nothing here exists on a public chain yet. See Part 10.

Part 10 — What's Next

Deploying to Sepolia (a public Ethereum testnet) is the next planned phase. Roughly: get an RPC endpoint and test ETH from a provider like Alchemy, fund a dedicated MetaMask account from a Sepolia faucet, store the RPC URL and a deploy private key via `npx hardhat keystore set` (the `sepolia` network entry already exists in `hardhat.config.ts`, referencing these via `configVariable(...)`), run the same `deploy-local.ts` script with `--network sepolia` (or an Ignition module), update `frontend/src/contracts.ts` with the new addresses, and deploy the frontend itself (Vercel is a natural fit for a Vite app). This moves the project from "only I can see this, on my own machine" to "anyone with a Sepolia-connected wallet can use this," which is the real test of whether it behaves like a genuine, if small, decentralized application.

A **"Remove Liquidity" UI** is a natural next frontend feature — the contract-side `Pair.burn` function (Part 4.6) is already deployed and fully functional; only the UI to call it (send LP tokens to the Pair, then call `burn`) is missing.

Further reading, roughly in order of how directly they extend this project: the original [Uniswap V2 whitepaper](#) (this project intentionally mirrors its core contracts, simplified); the [Solidity documentation](#) itself, especially its section on common vulnerabilities; and OpenZeppelin's contracts library, worth reading even just as source code, to see an audited, production-grade ERC-20 (and much more) implemented the way Part 2.2 gestured at.

Glossary

AMM (Automated Market Maker) — an exchange mechanism that prices trades algorithmically from pooled reserves, instead of matching individual buy/sell orders.

ABI (Application Binary Interface) — a JSON description of a contract's callable functions and events, used by external software to correctly encode calls and decode results.

Block — a batch of transactions plus a cryptographic pointer to the previous block.

Consensus — the mechanism by which decentralized nodes agree on one shared transaction history (Ethereum: proof-of-stake).

CREATE2 — an EVM opcode for deploying a contract to a deterministic, precomputable address based on the deployer, a chosen salt, and the contract's bytecode.

EOA (Externally Owned Account) — a blockchain account controlled by a private key (as opposed to a contract account, controlled by code).

ERC-20 — the standard interface for fungible tokens on Ethereum.

EVM (Ethereum Virtual Machine) — the deterministic runtime every Ethereum node uses to execute contract bytecode identically.

Gas — the fee, denominated in ETH, paid to have a transaction's computation executed and included in a block.

Gwei — one-billionth of an ETH (10^{-9} ETH); the usual unit for quoting gas prices.

Impermanent loss — the gap between a liquidity provider's pool position and simply holding the two underlying tokens, caused by price movement between them.

Liquidity pool — the shared reserve of two (or more) tokens an AMM trades against.

LP token — a token representing a liquidity provider's proportional share of a pool.

Mapping — Solidity's key → value storage type, akin to a hash table.

Modifier — reusable pre/post logic wrapped around a Solidity function (e.g., a reentrancy guard).

Private/public key — a private key is a secret number that lets you sign transactions; a public key (and, derived from it, an address) is safely shareable and lets others verify your signatures.

Reentrancy — a vulnerability where an external call mid-function lets another contract call back in before the first call finishes, seeing inconsistent state.

Slippage / price impact — the difference between a trade's expected and actual execution price, caused by the trade itself moving the pool's ratio.

Smart contract — immutable code deployed to and executed identically by every node on a blockchain.

Testnet — a public blockchain network (e.g., Sepolia) that mirrors mainnet's behavior using worthless test currency, for safe pre-production testing.

Transfer-then-call pattern — a design where a contract expects tokens to already be sent to its own address before a function is called, rather than pulling them via `transferFrom`; the function then compares its balance before and after to determine what arrived.

view / pure — Solidity function modifiers promising, respectively, no state changes, and no state changes *or* reads — both callable for free, without a transaction.

Wallet — software (e.g., MetaMask) that stores private keys and signs transactions on the user's behalf.